

We were asked to approve a Rp 50–90M logging fix.

THE RECOMMENDATION • APPROVE

Logging is the largest configuration-only saving we have. **Rp 81–105M a month, available in two weeks.**

THE REASONING • DO NOT APPROVE AS WRITTEN

The figure in the deck is stale, the named cause is unproven in production, and the real saving is somewhere else.

What was put in front of you, and what we did about it

One line in the platform decision deck asked for approval. We went and measured it against the billing data, the production logs and all 152 repositories.

THE CLAIM, AS WRITTEN

“Approve the Cloud Logging fix. It saves **Rp 50–90M a month** and is configuration only. That is more than retiring Bravo’s workflow tier would save.”

The mechanism named was `loggerLevel: full` on Feign, which makes every upstream call log its full request and response body.

WHAT WE CHECKED

FinOps billing for August 2026, complete month, by project, service and SKU. **Datadog production logs**, two days, every service. **All 152 repositories** under `squads/`, pulled to master on the day of writing — 149,729 files scanned.

Then, on a second pass, the **53 repositories the first cut left out**, plus 36 cloned for the purpose.

WHAT CAME BACK

The direction is right and the prize is bigger than claimed. But **three of the supporting facts do not survive contact with the data**, and the fix that was named is not the one that pays.

We also found a **live API credential sitting in production logs**, which nobody was looking for.

A CTO should not be asked to approve a number that cannot be traced to a measurement. Three of these now can, and one still cannot.

Logging costs Rp 636M a month, across three platforms

The deck counts one platform, and counts it low. August 2026, complete month, from the billing data.

CLOUD LOGGING · GCP Rp 271.8M 42.8% of the total. Up 38% since February. The only platform the deck mentions.	CORALOGIX Rp 253.3M 39.8%. Flat since February, billed as one marketplace charge. No cost document has ever asked what it is for.	DATADOG Rp 110.6M 17.4%. The platform we are standardising on. This is the baseline to protect.
--	--	--

THE FIGURES THAT NEED CORRECTING BEFORE THIS GOES FURTHER

CLAIM	DECK SAYS	MEASURED
Cloud Logging, estate — quoted in four documents and the CTO deck	Rp 140.5M	Rp 271.8M
Multiple of the workflow tier a migration would retire (≈Rp 58M)	2.4×	4.7×
Same multiple, counting all three log platforms	—	11×

THIS MAKES THE ARGUMENT STRONGER, NOT WEAKER

The point the deck was making — that configuration work beats migration work — holds more firmly at Rp 636M than at Rp 140.5M. **We are asking to fix the evidence, not to drop the recommendation.**

A third of it is spent on environments with no customers

PROJECT	AUGUST 2026	VOLUME
bravo-project-331802 – production	Rp 125,414,087	18.05 TB
bravo-project-nonprod – dev, SIT, UAT	Rp 56,360,125	7.66 TB
bfi-devsecops – CI/CD	Rp 34,898,639	4.77 TB
bfi-internal-app-production	Rp 9,012,031	1.23 TB
bfi-internal-app-nonprod	Rp 1,564,409	0.21 TB
core-system-nonprod	Rp 236,516	0.03 TB
Non-production, added up	Rp 93,059,689	12.7 TB

34%

OF THE CLOUD LOGGING BILL IS DEV, SIT, UAT AND CI. NO CUSTOMER IS SERVED BY ANY OF IT. THIS IS THE LARGEST CERTAIN SAVING IN THE PACK.

TWO LINES THAT BELONG TO NOBODY’S BACKLOG

Cloud SQL audit and slow-query logs in non-production — Rp 18.4M a month. That is three times what the production database logs cost.

VPC flow logs — Rp 7.9M a month. Sampling is almost certainly at 100%. Dropping to 10% cuts it to under Rp 1M with no practical loss.

None of this needs a squad, a release, or a line of code. It is a retention setting and an exclusion filter.

The fix that was named cannot be shown to be doing anything

This is the part that should not go to a board without being settled. It takes one hour.

WHAT THE CODE SAYS — THE CLAIM IS REAL IN THE SOURCE

In `bravo-bpm-service`, `application.yaml` sets `loggerLevel: full` on 129 Feign entries including the default, and puts `com.bfi.bravo.adapter` at **DEBUG**. All 57 Feign clients live under that package. `application-prod.yaml` sets `root: INFO` but never overrides it — different keys in Spring Boot, so **DEBUG** survives.

On paper, every call to PEFINDO, SLIK, Dukcapil and CONFINS logs its full body.

WHAT PRODUCTION SAYS — NOTHING

Across two days, every production service:

Logs at `status:debug` — 0.

Logs containing `END HTTP`, the marker Feign writes at the end of every full-logged exchange — 0.

TWO EXPLANATIONS, AND WE CANNOT TELL WHICH

1. A deployment environment variable overrides the log level. None of the 152 repositories contains a deployment manifest — they live in a GitOps repo we do not have. This is the likelier one.

2. Datadog drops **DEBUG** before indexing, and Cloud Logging still pays for it.

THE ASK

Read the `ms-bpm` deployment manifest. **One hour.** It decides whether this fix is worth Rp 0 or Rp 40M a month.

Until that is done, the Rp 50–90M figure is a code reading with no production signal behind it. It should not be quoted as settled.

THE CONFIGURATION IS STILL WRONG, WHATEVER THE ANSWER

A default of `full` means one environment variable — or one careless merge — starts writing customer credit-bureau payloads into logs. **That is a data-protection exposure sitting behind a single line.** Six repositories carry it. Fix it regardless of whether it is currently costing money.

Five services produce 64% of every log line we pay for

Measured in production, not read from code. Almost none of it is telemetry anyone reads.

confins-prod-ms-lms-ar-be	2,640,747	29%
Genuinely blank log lines. Empty message field, several a second, 9.25 million a week. Shipped, indexed and stored. This service has no identified owner.		
prod-lora-task	1,291,274	14%
3.8 million warnings a week. A browser tab closing. An optional form field being absent. Nobody will ever act on any of it.		
prod-ms-calculation	770,394	9%
The entire HTTP error object serialised on every 4xx — roughly 12,000 a day. See the next slide.		
prod-ms-cnv	752,562	8%
The same error 20,000 times a day. Employee access records failing to sync and being discarded. A real defect nobody noticed, because it is buried in the noise.		
prod-ms-bpm	387,332	4%
Split JSON fragments and stack traces logged one frame per entry.		
Top five of 71 services	5,842,309	64%

THE DISCOVERY THAT EXPLAINS THE REST

The container runtime **splits any log line over 16 KB** into separate chunks, which arrive as unrelated entries. Neither half is valid JSON, so neither parses.

93.6% of our production logs cannot be parsed at all — 8.46 million of 9.03 million over two days. Large log lines are not just expensive. They are destroying the log pipeline.

WHY THIS MATTERS MORE THAN THE FEIGN QUESTION

These five are measured, owned and fixable. Between them they are worth more than the disputed fix, and the `ms-cnv` item is a **correctness defect** — 20,000 employee access updates a day are being thrown away, possibly including role revocations.

No monitor noticed, because **no monitor watches an outcome**.

A live API credential is being written to production logs

Found while measuring cost. It is not a cost item and it should not wait for this programme.

ONE LINE, IN ONE FILE

`lms-calculation-service/src/helpers/HttpHelper.ts:27` runs `JSON.stringify(error)` on every failed HTTP call and logs the result.

On an Axios error that serialises the entire request config — headers and body included.

WHAT IS IN EACH ENTRY, CONFIRMED IN PRODUCTION

- the `api-secret` header in plaintext, in full, on every 4xx or 5xx
- the request body, including customer `birth_date`
- the full stack trace
- terminal colour codes, which is why these are filed as `info` and never reach an error dashboard

12,000

ENTRIES A DAY CARRYING THE CREDENTIAL. 437 MEASURED IN A SINGLE 51-MINUTE WINDOW. IT HAS BEEN THERE FOR AS LONG AS THIS CODE HAS SHIPPED.

WHAT WE ARE ASKING FOR

1. Replace the interceptor with four structured fields. Two hours.
2. **Rotate the secret.** Treat it as leaked. It is shared with `prod-ms-insurance`.
3. Tell whoever owns security review.

Anyone with log read access in the production project has had this credential.

This was not what we were looking for. It is what you find when you read nine million log lines instead of counting them.

We then read the other 53 repositories. It was not the same problem.

The first pass ranked repositories by a code scan. That scan was Java-shaped, and about two-thirds of the estate is Go — those repositories scored zero on it while carrying four of the five worst severity mixes in production.

bau-prod-ms-otrs-report	live now	rotate
Google Chat webhook credentials in the log index. The key and token are in the URL query string, and the URL is logged on every message sent. Anyone with log read access can post into those spaces.		
prod-ms-employee	907,511	a week
Whole HR records at <code>info</code> . Religion, marital status, date and place of birth, bank account number and holder name, personal email, mobile number — on every inbound and outbound message.		
prod-ms-notification	not firing	present
The JWT signing key, printed with <code>fmt.Printf</code> — which bypasses the logger, so no level could ever suppress it. We checked: that path is not being exercised today. The code is there.		
prod-ms-auth	98.5%	error
16,934 errors and 3 info entries a week, because one function logs at <code>error</code> whatever status code it is handed. A wrong OTP was an error.		

THE PATTERN UNDERNEATH

The Go estate’s masking is **wired up and empty**. Twenty-two repositories hand the scrubber a field list that defaults to nothing. In `lora-gateway-service` the scrubber call is **commented out**, under a comment reading “*NON PRODUCTION ENVIRONMENT ONLY*”. It writes 69,433 bodies a week in production.

Reading the code tells you masking is on. It is not.

WHAT WE HAD WRONG

Our own earlier note called the API gateway’s million log lines “*an access log wearing the wrong level. Nothing is wrong.*” **Both halves were wrong.**

The gateway levels by status code, correctly. Its 942,909 warnings a week are **942,909 real 4xx responses** — 415,388 of them one WhatsApp endpoint returning 400. It is the only thing reporting them. Filtering it would destroy the evidence.

We own more of Datadog than we use, and parts of it are broken

We audited every environment. The expensive half is already deployed and working. The cheap half is missing.

ALREADY WORKING, AND BETTER THAN OUR OWN DOCUMENTS CLAIM

APM covers 42 production services — 79 million spans over two days, including database and queue spans. Most squads do not know this exists. `prod-ms-bpm` alone produces over three million spans a week — controllers, JPA and outbound calls. *What it does not produce is a Camunda span: no process instance, no service task. That gap is real and is [bravo-observability.md](#) §2.*

LOGS AND TRACES ARE NOT JOINED UP

Zero logs carry a trace id. Nobody can click from a slow trace to the logs for that request, or from an error to the trace that caused it.

Only **30 of 71 services send both** logs and traces. Two services have *different names* in logs and traces, so Datadog treats them as four.

MONITORS THAT REPORT OK AND CAN NEVER FIRE

A family of about twenty log monitors targets `dev-`, `sit-` and `uat-` services. **No non-production logs reach Datadog at all**, so none of them can ever trigger.

Worse, the production sharia monitors query `prod-sharia-sharia-bpm-sharia` — note the **doubled word**, a service that does not exist. **They show OK because nothing matches, not because nothing is wrong.**

THREE MORE, BRIEFLY

8 synthetic tests, all DNS and SSL checks. Nothing tests whether a customer can submit a loan.

23 Dockerfiles download the tracer from `releases/latest` — unpinned, in production.

Production alerts route to `@googlechat-test` and `@webhook-testwebhook`.

The order matters, because Datadog bills on what we send it

The obvious move is to turn on the Datadog features now. That would make the bill worse.

THE TRAP WE NEARLY WALKED INTO

The fix for 16 KB line splitting is to have the Datadog agent rejoin the fragments. Correct — but applied to *today’s* stream it would faithfully reassemble the 64 KB request bodies and the credential-carrying error dumps into **single large billable events**, and index them.

We would pay Datadog premium rates to reconstruct exactly the content that should never have been logged.

Clean the stream first. Then enable, on half the volume, at half the recurring price.

THE GATE SRE HOLDS BEFORE ENABLING ANYTHING

PRODUCTION LOG LINES / DAY

4.5M → under 2.5M

Protects the per-GB price of everything downstream. The five services on slide 06 are most of the gap.

SHARE DATADOG CAN PARSE

6.4% → over 80%

The one people will want to skip. Datadog cost filters work on **parsed attributes** — on raw text they are guesswork, and will silently drop evidence we needed.

SQUADS FIRST, SRE SECOND

This is not a queue. The squad work is what makes the SRE work affordable, and it is already written up per repository — nineteen files, each naming the file and line to change.

The squad work is written. What is left is review and merge.

Phase 2 was a plan when this deck was first drafted. It is now 64 open pull requests, one per repository, each with a file naming the finding, the production numbers and the change.

73

REPOSITORIES READ LINE BY LINE, AGAINST PRODUCTION EVIDENCE

64

PULL REQUESTS OPEN, BRANCH `FIX/LOGGING` IN EACH

NINE DEFECTS, AND NONE OF THEM FOUND BY READING

The Go changes are now **compiled, formatted and linted** — `go build`, `gofmt` and `golangci-lint` against each repository’s own config. CI found five defects, the local toolchain another four: a parameter named `logger` shadowing the logger package in seven repositories, two `gofmt` rules, an unused import, `gochecknoglobals`, `tagalign`, and a comment block missing its `//`.

Every one was invisible to reading. Java still cannot be built here — no Maven, no JVM — so those pull requests remain unverified.

ONE HOUR OF PLATFORM WORK IS HOLDING UP SIX WEEKS OF SQUAD WORK

A job named `Static Analysis – SonarQube` in the shared CI workflow fails at its Codacy step:

Invalid configuration: Either a project or account API token must be provided

It red-flags 23 of the 64 pull requests for a reason unrelated to their content. Confirmed identical on three repositories whose changes are a struct-tag default and two log levels.

Until it is fixed, every reviewing squad sees a red check and has to be told to ignore it.

Analysis is not the constraint any more. A missing CI token and squad review time are.

Four phases. The money is in the first one.

PHASE 1 • NOW Platform configuration and defect fixes Non-production exclusion filters and retention. Cloud SQL logs off in non-prod. Flow log sampling to 10%. Fix the dead monitors. Pin the tracer. Name an owner for the blank-line service. NO DATADOG COST • 2 WEEKS	PHASE 2 • NEXT Squads cut log volume Seventy-three per-repository files and 64 open pull requests, each naming the file and line. The five services on slide 06 are 64% of it. SRE holds the gate and tracks two numbers weekly. REDUCES COST • 4-6 WEEKS	PHASE 3 • GATED Ingestion and correlation Line reassembly, trace correlation, per-service cost filters in the same change. Trace the API gateway, with sampling from day one. SMALL INCREASE • 1 WEEK	PHASE 4 • ONE AT A TIME Paid add-ons Runtime metrics, then database monitoring on ten services, then profiling on one. Real synthetic tests. SIT and UAT logs into Datadog — last, and errors only. PER PRODUCT • MEASURE EACH
---	---	--	---

Service	Cost
S3	105M
Lambda	81M
DynamoDB	2M

Rp 81–105M

A MONTH, RECOVERED IN PHASE 1 ALONE. NO CODE CHANGE, NO SQUAD BACKLOG, NO RELEASE.

2 wks

TO BANK IT. PHASE 1 IS PLATFORM CONFIGURATION AND ONE ACCESS REVIEW.

WHAT WE ARE DELIBERATELY NOT DOING YET

Sending non-production logs to Datadog. It is the item most likely to blow the budget, and it is a convenience improvement with a real recurring cost. It comes last, restricted to SIT and UAT, errors only, three-day retention — and only after the savings are banked.

Coralogix stays an open question. Rp 253M a month, flat since February, and no document we hold explains what it is for. That answer may be worth more than everything else on this slide.

Six things we need from you

1	Approve the logging programme, on the corrected reasoning. Rp 81-105M a month from platform configuration, available in two weeks, with no code change. The direction in the original recommendation was right; the numbers behind it were not.	APPROVE TODAY
2	Treat the leaked credentials as an incident, not a backlog item. Rotate the API secret shared with prod-ms-insurance and fix HttpHelper.ts . Then rotate the Google Chat webhooks logged by bau-prod-ms-otrs-report — their key and token are in the log index now, and nobody on our side has write access to that repository.	DIRECT TODAY
3	Order the one-hour manifest check before any of this is quoted externally. It decides whether the Feign fix is worth Rp 0 or Rp 40M a month. Until it is done, that figure should not appear in a board pack.	DIRECT THIS WEEK
4	Hold the sequencing. Squad cleanup gates SRE enablement, on two published numbers. If that gate is waived under delivery pressure, we move the waste from Cloud Logging into Datadog and the bill goes up instead of down.	HOLD THE LINE
5	Name an owner for confins-prod-ms-lms-ar-be . It is 29% of all production log volume, it emits blank lines, and no squad in our mapping claims it. Nothing else in Phase 2 is worth as much per hour of work.	ASSIGN THIS WEEK
6	Ask someone to justify Coralogix. Rp 253M a month, flat since February, for a third logging platform alongside Cloud Logging and Datadog. We are not recommending cancelling it. We are saying nobody has been asked.	ASK THIS MONTH

WHAT IS ALREADY WRITTEN AND WAITING

logging-cost.md — the cost analysis and the corrections owed to four other documents, one deck and itself. squads-guide.md — the rules, now covering the Go estate as well as the Java one. sre-datadog-recommendations.md — the configuration audit and the phased plan. Seventy-three per-repository files and 64 open pull requests. No further analysis is needed to start Phase 1 or Phase 2.